

#Exercise 2 — DCGAN using Fashion-MNIST

Problem statement

Implement a Deep Convolutional Generative Adversarial Network (DCGAN) using the Fashion-MNIST dataset to generate synthetic grayscale clothing images.

Loading and normalizing Fashion-MNIST.

Building a Generator using transposed convolution.

Building a Discriminator using strided convolution.

Training the Generator and Discriminator alternately.

Generating image grids at epochs 1, 5, 10, and 20.

Plotting Generator and Discriminator losses.

Experimenting with a hyperparameter.

Analysing the generated images.

Step 1: Import libraries

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
import time

print("TensorFlow version:", tf.__version__)
print("GPU available:", tf.config.list_physical_devices("GPU"))

TensorFlow version: 2.20.0
GPU available: []
```

Fashion-MNIST is available through Keras, so no separate dataset download or manual file upload is required. The first run requires internet access to retrieve the dataset if it is not already cached.

Step 2: Load and preprocess Fashion-MNIST

Fashion-MNIST contains 28×28 grayscale clothing images across 10 categories. We only need the images, not their class labels, because this is an unconditional GAN.

```
# Load Fashion-MNIST
(x_train, _), (_, _) = tf.keras.datasets.fashion_mnist.load_data()

# Add channel dimension: (N, 28, 28) -> (N, 28, 28, 1)
x_train = np.expand_dims(x_train, axis=-1)
```

```
# Normalize pixel values from [0, 255] to [-1, 1]
x_train = (x_train.astype("float32") - 127.5) / 127.5

print("Dataset shape:", x_train.shape)
print("Minimum pixel value:", x_train.min())
print("Maximum pixel value:", x_train.max())

Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 _____ 0s 0us/step
Dataset shape: (60000, 28, 28, 1)
Minimum pixel value: -1.0
Maximum pixel value: 1.0
```

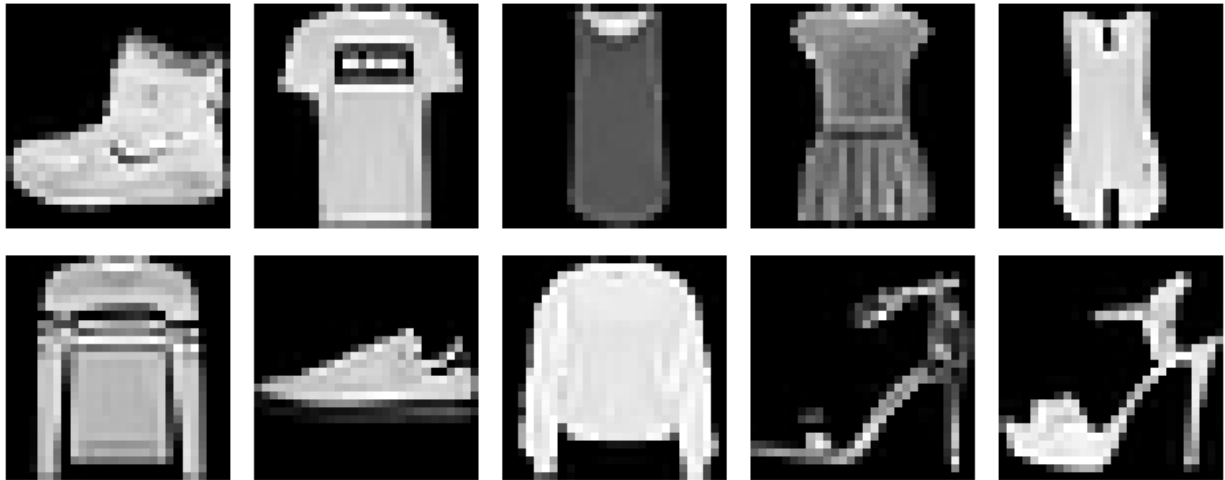
Visualize some real images:

```
plt.figure(figsize=(8, 4))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i, :, :, 0], cmap="gray", vmin=-1, vmax=1)
    plt.axis("off")

plt.suptitle("Real Fashion-MNIST Images")
plt.tight_layout()
plt.show()
```

Real Fashion-MNIST Images



Step 3: Build the Generator

The Generator converts a random latent vector into a synthetic 28×28 grayscale image.

Architecture:

Random noise: 100 values.

Dense layer: projects the noise into a feature representation.

Reshape: converts the vector into a 7×7×128 feature map.

Transposed convolution: upsamples to 14×14.

Transposed convolution: upsamples to 28×28.

Tanh activation: produces the final normalized image.

```
LATENT_DIM = 100

def build_generator():
    model = models.Sequential(name="Generator")

    model.add(layers.Input(shape=(LATENT_DIM,)))

    # Project latent vector into an initial feature map
    model.add(layers.Dense(7 * 7 * 128, use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.ReLU())

    model.add(layers.Reshape((7, 7, 128)))

    # Upsample: 7x7 -> 14x14
    model.add(layers.Conv2DTranspose(
```

```

        64,
        kernel_size=4,
        strides=2,
        padding="same",
        use_bias=False
    ))
    model.add(layers.BatchNormalization())
    model.add(layers.ReLU())

    # Upsample: 14x14 -> 28x28
    model.add(layers.Conv2DTranspose(
        1,
        kernel_size=4,
        strides=2,
        padding="same",
        activation="tanh"
    ))

    return model

```

```

generator = build_generator()
generator.summary()

```

Model: "Generator"

Layer (type) Param #	Output Shape	
dense (Dense) 627,200	(None, 6272)	
batch_normalization (BatchNormalization) 25,088	(None, 6272)	
re_lu (ReLU) 0	(None, 6272)	
reshape (Reshape) 0	(None, 7, 7, 128)	

131,072	conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 64)	
256	batch_normalization_1 (BatchNormalization)	(None, 14, 14, 64)	
0	re_lu_1 (ReLU)	(None, 14, 14, 64)	
1,025	conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 1)	

Total params: 784,641 (2.99 MB)

Trainable params: 771,969 (2.94 MB)

Non-trainable params: 12,672 (49.50 KB)

Step 4: Build the Discriminator

The Discriminator receives an image and produces a single score indicating how real it considers the image to be.

Architecture:

Input: 28×28×1.

First convolution: downsamples to 14×14.

Second convolution: downsamples to 7×7.

Flatten: converts feature maps into a vector.

Dense layer: produces one real/fake logit.

```
def build_discriminator():
    model = models.Sequential(name="Discriminator")

    model.add(layers.Input(shape=(28, 28, 1)))

    # Downsample: 28x28 -> 14x14
```

```

model.add(layers.Conv2D(
    64,
    kernel_size=4,
    strides=2,
    padding="same"
))
model.add(layers.LeakyReLU(negative_slope=0.2))
model.add(layers.Dropout(0.3))

# Downsample: 14x14 -> 7x7
model.add(layers.Conv2D(
    128,
    kernel_size=4,
    strides=2,
    padding="same"
))
model.add(layers.LeakyReLU(negative_slope=0.2))
model.add(layers.Dropout(0.3))

model.add(layers.Flatten())

# Output a logit, not a probability
model.add(layers.Dense(1))

return model

```

```

discriminator = build_discriminator()
discriminator.summary()

```

Model: "Discriminator"

Layer (type) Param #	Output Shape
conv2d (Conv2D) 1,088	(None, 14, 14, 64)
leaky_re_lu (LeakyReLU) 0	(None, 14, 14, 64)
dropout (Dropout) 0	(None, 14, 14, 64)

conv2d_1 (Conv2D)	(None, 7, 7, 128)	
131,200		
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	
0		
dropout_1 (Dropout)	(None, 7, 7, 128)	
0		
flatten (Flatten)	(None, 6272)	
0		
dense_1 (Dense)	(None, 1)	
6,273		

Total params: 138,561 (541.25 KB)

Trainable params: 138,561 (541.25 KB)

Non-trainable params: 0 (0.00 B)

The final layer does not use a sigmoid activation. We will use binary cross-entropy with `from_logits=True`, which combines the sigmoid calculation and loss computation in a numerically stable way.

Step 5: Test the network dimensions

Before training, verify that both networks work correctly.

```
# Test Generator
noise = tf.random.normal([1, LATENT_DIM])
generated_image = generator(noise, training=False)

print("Generated image shape:", generated_image.shape)

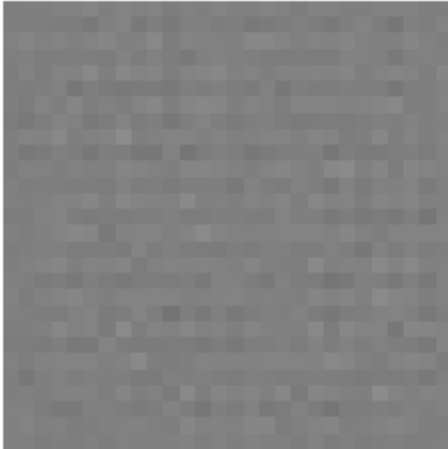
# Test Discriminator
score = discriminator(generated_image, training=False)
print("Discriminator output shape:", score.shape)

Generated image shape: (1, 28, 28, 1)
Discriminator output shape: (1, 1)
```

Visualize an initial, untrained generated image:

```
plt.figure(figsize=(3, 3))
plt.imshow(generated_image[0, :, :, 0], cmap="gray", vmin=-1, vmax=1)
plt.title("Generated Image Before Training")
plt.axis("off")
plt.show()
```

Generated Image Before Training



Step 6: Define the loss functions and optimizers

The Discriminator learns to distinguish real images from generated images. The Generator learns to produce images that the Discriminator classifies as real.

```
# Binary cross-entropy loss
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(
        tf.ones_like(real_output), real_output
    )

    fake_loss = cross_entropy(
        tf.zeros_like(fake_output), fake_output
    )

    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(
        tf.ones_like(fake_output), fake_output
    )

# Optimizers
LEARNING_RATE = 0.0002
```

```

generator_optimizer = tf.keras.optimizers.Adam(
    learning_rate=LEARNING_RATE,
    beta_1=0.5
)

discriminator_optimizer = tf.keras.optimizers.Adam(
    learning_rate=LEARNING_RATE,
    beta_1=0.5
)

print("Loss functions and optimizers are ready.")

```

Loss functions and optimizers are ready.

Why use beta_1=0.5? This is a commonly used DCGAN configuration for Adam that changes the momentum behaviour compared with its default setting.

Step 7: Prepare the training dataset

```

BUFFER_SIZE = len(x_train)
BATCH_SIZE = 128
EPOCHS = 20

train_dataset = (
    tf.data.Dataset.from_tensor_slices(x_train)
    .shuffle(BUFFER_SIZE)
    .batch(BATCH_SIZE, drop_remainder=True)
    .prefetch(tf.data.AUTOTUNE)
)

print("Batch size:", BATCH_SIZE)
print("Epochs:", EPOCHS)
print("Batches per epoch:", BUFFER_SIZE // BATCH_SIZE)

```

Batch size: 128
Epochs: 20
Batches per epoch: 468

Step 8: Implement the training step

During each batch:

Generate random noise.

Generate fake images.

Calculate the Discriminator's loss on real and fake images.

Update the Discriminator.

Generate another batch of fake images.

Calculate the Generator's loss.

Update the Generator so that the fake images are more likely to be classified as real.

```
@tf.function
def train_step(real_images):
    batch_size = tf.shape(real_images)[0]

    # Generate random latent vectors
    noise = tf.random.normal([batch_size, LATENT_DIM])

    # -----
    # Train the Discriminator
    # -----
    with tf.GradientTape() as disc_tape:
        fake_images = generator(noise, training=True)

        real_output = discriminator(real_images, training=True)
        fake_output = discriminator(
            tf.stop_gradient(fake_images),
            training=True
        )

        d_loss = discriminator_loss(real_output, fake_output)

    disc_gradients = disc_tape.gradient(
        d_loss, discriminator.trainable_variables
    )

    discriminator_optimizer.apply_gradients(
        zip(disc_gradients, discriminator.trainable_variables)
    )

    # -----
    # Train the Generator
    # -----
    noise = tf.random.normal([batch_size, LATENT_DIM])

    with tf.GradientTape() as gen_tape:
        fake_images = generator(noise, training=True)
        fake_output = discriminator(fake_images, training=True)

        g_loss = generator_loss(fake_output)

    gen_gradients = gen_tape.gradient(
        g_loss, generator.trainable_variables
    )

    generator_optimizer.apply_gradients(
        zip(gen_gradients, generator.trainable_variables)
    )
```

```
return g_loss, d_loss
```

Notice that `tf.stop_gradient()` prevents the Discriminator's training step from updating the Generator. The Generator is updated separately in its own training step.

Step 9: Generate and save image grids

We will use the same fixed set of latent vectors throughout training. This allows us to compare how the generated images change over time.

```
NUM_EXAMPLES = 16

fixed_noise = tf.random.normal([NUM_EXAMPLES, LATENT_DIM])

def generate_and_save_images(model, epoch):
    predictions = model(fixed_noise, training=False)

    fig, axes = plt.subplots(4, 4, figsize=(6, 6))

    for i, ax in enumerate(axes.flat):
        image = predictions[i, :, :, 0].numpy()

        ax.imshow(image, cmap="gray", vmin=-1, vmax=1)
        ax.axis("off")

    fig.suptitle(f"DCGAN Generated Images - Epoch {epoch}")
    plt.tight_layout()
    plt.savefig(f"drgan_epoch_{epoch:02d}.png", dpi=150)
    plt.show()
    plt.close(fig)
```

Step 10: Train the DCGAN for 20 epochs

```
generator_losses = []
discriminator_losses = []

epochs_to_visualize = {1, 5, 10, 20}

for epoch in range(1, EPOCHS + 1):
    start_time = time.time()

    epoch_g_losses = []
    epoch_d_losses = []

    for real_images in train_dataset:
        g_loss, d_loss = train_step(real_images)

        epoch_g_losses.append(float(g_loss.numpy()))
        epoch_d_losses.append(float(d_loss.numpy()))
```

```

avg_g_loss = np.mean(epoch_g_losses)
avg_d_loss = np.mean(epoch_d_losses)

generator_losses.append(avg_g_loss)
discriminator_losses.append(avg_d_loss)

elapsed = time.time() - start_time

print(
    f"Epoch {epoch:02d}/{EPOCHS} | "
    f"Generator Loss: {avg_g_loss:.4f} | "
    f"Discriminator Loss: {avg_d_loss:.4f} | "
    f"Time: {elapsed:.1f}s"
)

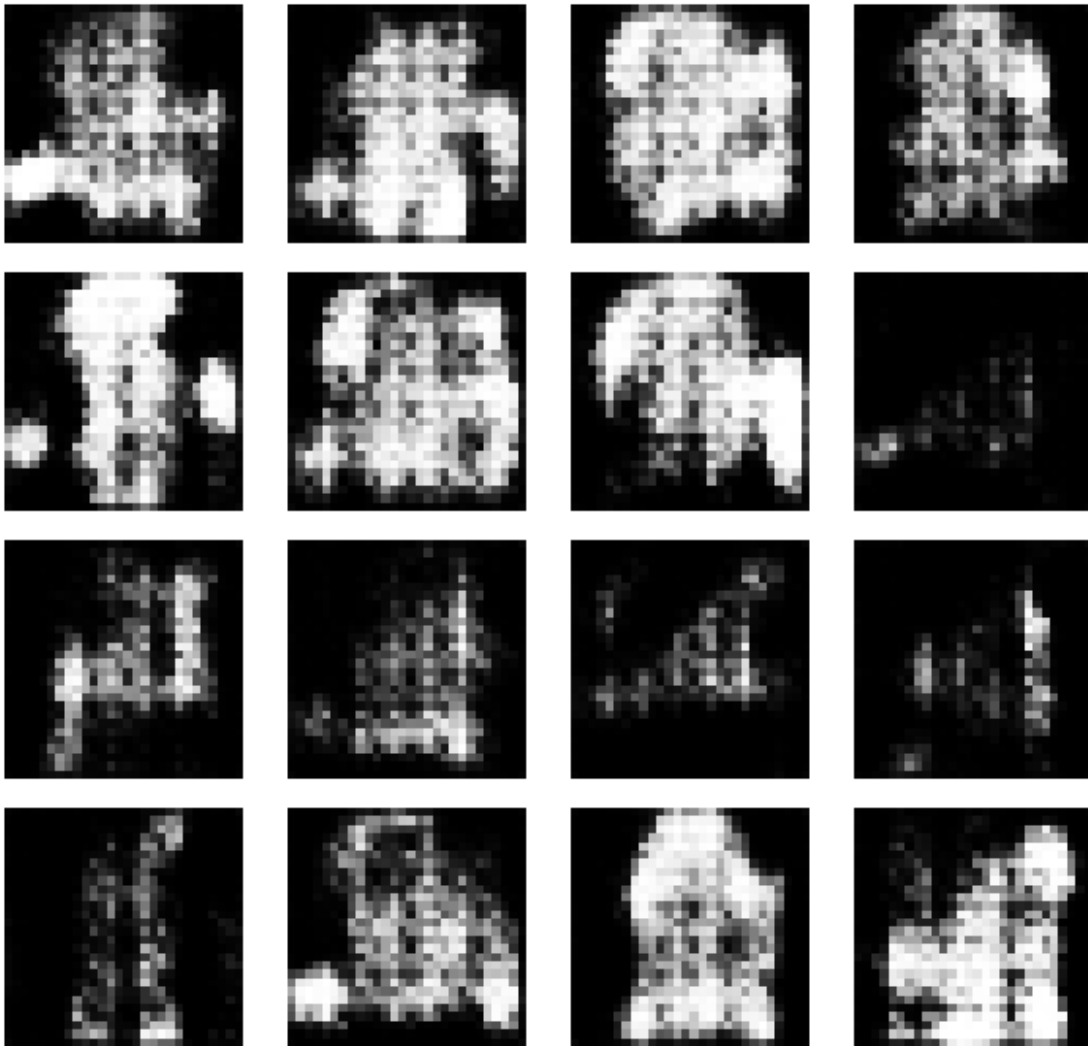
if epoch in epochs_to_visualize:
    generate_and_save_images(generator, epoch)

print("DCGAN training completed.")

```

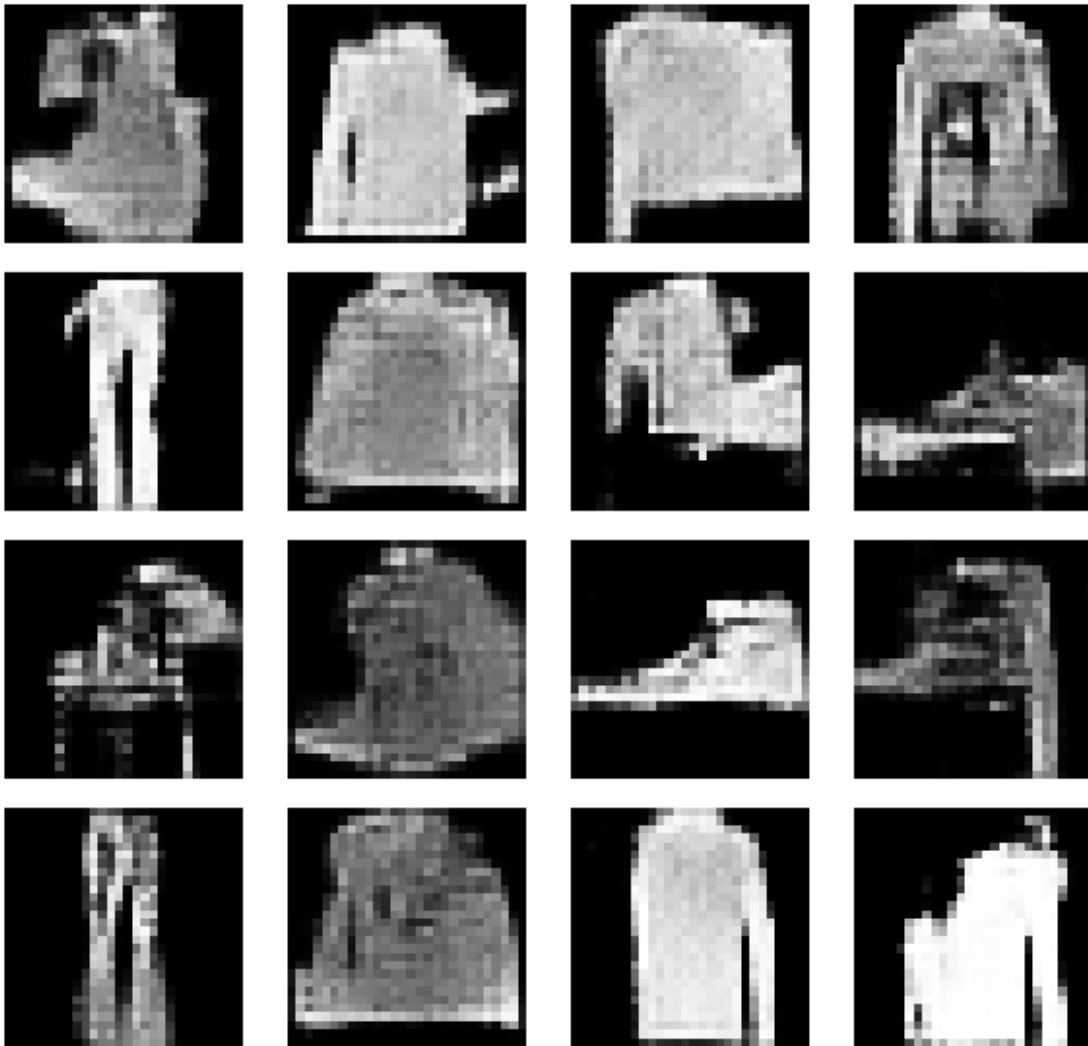
Epoch 01/20 | Generator Loss: 0.8129 | Discriminator Loss: 1.2407 |
Time: 420.6s

DCGAN Generated Images - Epoch 1



Epoch 02/20		Generator Loss: 0.9128		Discriminator Loss: 1.1950	
Time: 381.9s					
Epoch 03/20		Generator Loss: 0.9718		Discriminator Loss: 1.1570	
Time: 325.8s					
Epoch 04/20		Generator Loss: 0.8916		Discriminator Loss: 1.2291	
Time: 323.4s					
Epoch 05/20		Generator Loss: 0.8596		Discriminator Loss: 1.2580	
Time: 322.7s					

DCGAN Generated Images - Epoch 5



```
Epoch 06/20 | Generator Loss: 0.8433 | Discriminator Loss: 1.2754 |  
Time: 323.0s  
Epoch 07/20 | Generator Loss: 0.8254 | Discriminator Loss: 1.2936 |  
Time: 324.1s  
Epoch 08/20 | Generator Loss: 0.8121 | Discriminator Loss: 1.3064 |  
Time: 321.9s  
Epoch 09/20 | Generator Loss: 0.8041 | Discriminator Loss: 1.3093 |  
Time: 329.9s  
Epoch 10/20 | Generator Loss: 0.8085 | Discriminator Loss: 1.3121 |  
Time: 334.3s
```

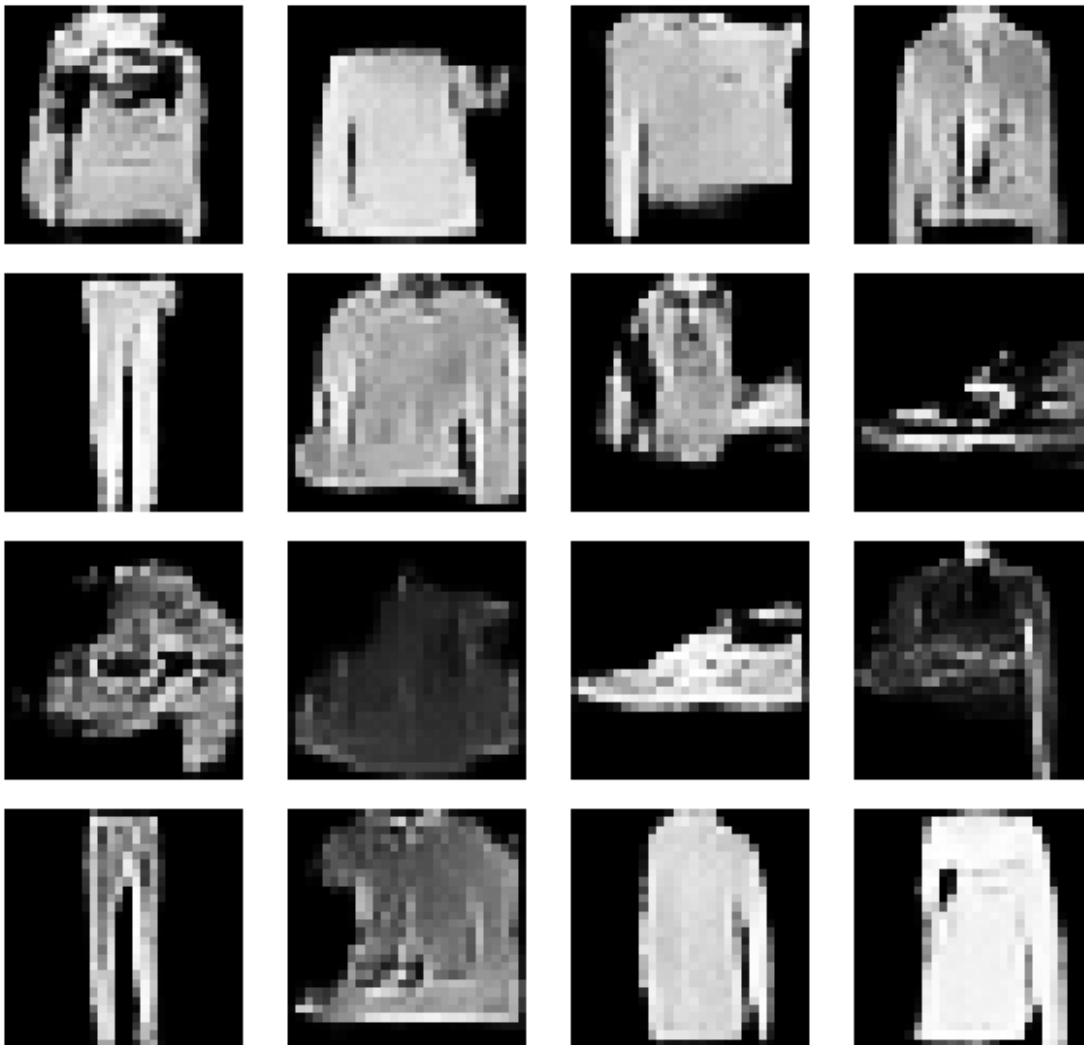
DCGAN Generated Images - Epoch 10



Epoch 11/20		Generator Loss: 0.8079		Discriminator Loss: 1.3132	
Time: 327.1s					
Epoch 12/20		Generator Loss: 0.8075		Discriminator Loss: 1.3149	
Time: 327.0s					
Epoch 13/20		Generator Loss: 0.8033		Discriminator Loss: 1.3142	
Time: 339.1s					
Epoch 14/20		Generator Loss: 0.8035		Discriminator Loss: 1.3177	
Time: 349.3s					
Epoch 15/20		Generator Loss: 0.7991		Discriminator Loss: 1.3168	
Time: 346.9s					
Epoch 16/20		Generator Loss: 0.7981		Discriminator Loss: 1.3189	
Time: 333.8s					
Epoch 17/20		Generator Loss: 0.7987		Discriminator Loss: 1.3233	
Time: 330.0s					
Epoch 18/20		Generator Loss: 0.7944		Discriminator Loss: 1.3183	

```
Time: 320.0s
Epoch 19/20 | Generator Loss: 0.7967 | Discriminator Loss: 1.3229 |
Time: 317.3s
Epoch 20/20 | Generator Loss: 0.7953 | Discriminator Loss: 1.3229 |
Time: 318.3s
```

DCGAN Generated Images - Epoch 20



DCGAN training completed.

Step 11: Plot Generator and Discriminator losses

```
plt.figure(figsize=(10, 5))
plt.plot(
    range(1, EPOCHS + 1),
```

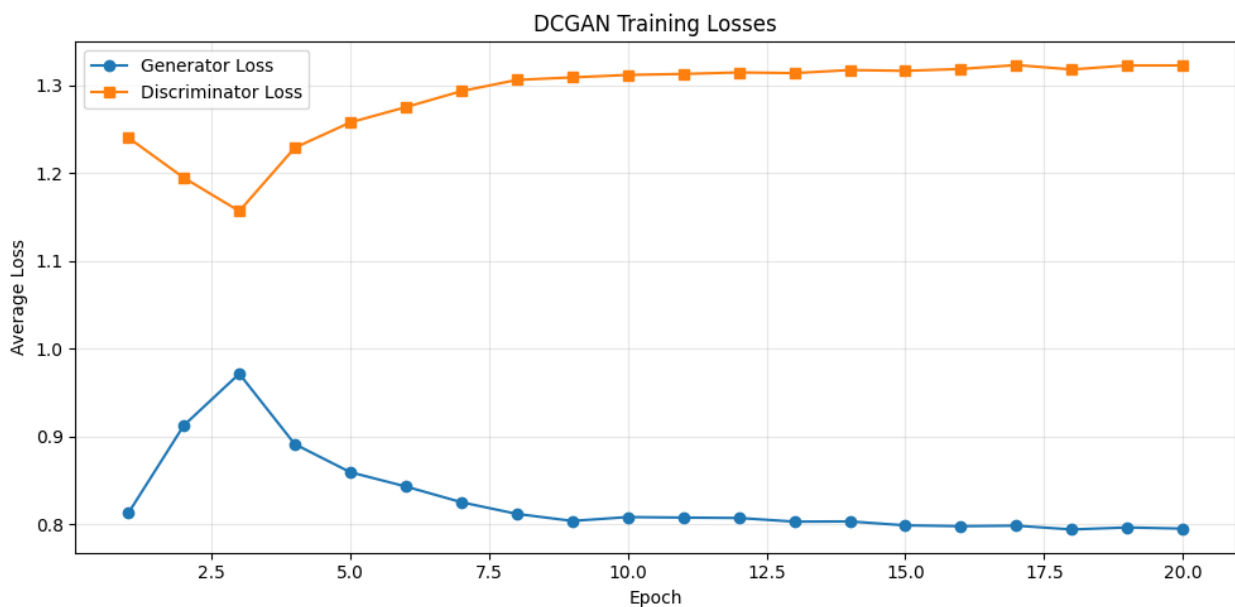
```

    generator_losses,
    marker="o",
    label="Generator Loss"
)

plt.plot(
    range(1, EPOCHS + 1),
    discriminator_losses,
    marker="s",
    label="Discriminator Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Average Loss")
plt.title("DCGAN Training Losses")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("dcgan_training_losses.png", dpi=150)
plt.show()

```



How to interpret the graph

Generator loss measures how poorly the Generator fools the Discriminator under the chosen loss formulation.

Discriminator loss measures how well the Discriminator distinguishes real images from generated images.

The losses may fluctuate because both networks are learning against each other.

Neither loss must decrease monotonically.

Low loss alone does not guarantee good image quality.

A Discriminator that becomes too strong can make Generator training difficult. Conversely, a Generator may produce repetitive images even when the loss values appear reasonable.

Step 12: Compare the generated images at different epochs

The training loop automatically displays generated images at epochs 1, 5, 10, and 20 and saves them as PNG files.

You can also display the saved images together for comparison.

```
from PIL import Image
import os

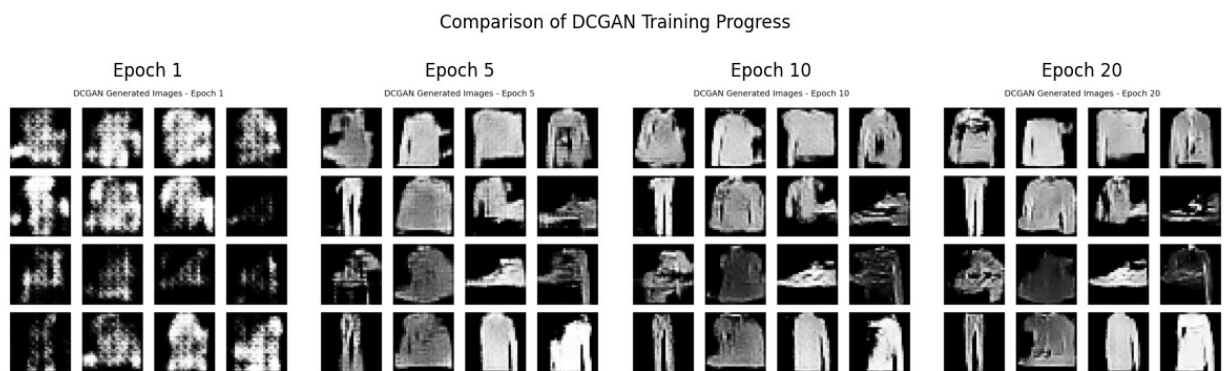
epochs_to_compare = [1, 5, 10, 20]

fig, axes = plt.subplots(1, 4, figsize=(12, 4))

for ax, epoch in zip(axes, epochs_to_compare):
    filename = f"drgan_epoch_{epoch:02d}.png"

    if os.path.exists(filename):
        image = Image.open(filename)
        ax.imshow(image)
        ax.set_title(f"Epoch {epoch}")
        ax.axis("off")

plt.suptitle("Comparison of DCGAN Training Progress")
plt.tight_layout()
plt.show()
```



Note: Each saved PNG contains a 4×4 grid, so this comparison displays the full saved grids at a smaller scale.

Step 13: Experiment with a different learning rate

For the required experimentation task, compare the baseline learning rate of 0.0002 with 0.0001.

Important: To make a fair comparison, each configuration should start from newly initialized networks and optimizers. Do not simply change the learning rate halfway through the original training and treat it as an independent experiment.

You can rerun the notebook from the model-building cells after changing:

Then rebuild the Generator and Discriminator, recreate both optimizers, and repeat the training process for 20 epochs. Record the loss curves and generated image grids for each run.

Step 14: Analysis and interpretation

EPOCH 1

EPOCH5

EPOCH10

EPOCH20

1. Why does a convolutional architecture help when generating images?
2. What happens to spatial dimensions in the Generator and Discriminator?
3. Why should the real images and Generator output use compatible pixel ranges?
4. How does the Discriminator's feedback improve the Generator?
5. Why might generated images remain blurry or unrecognizable after training?
6. What do the Generator and Discriminator loss curves tell us, and why should they not be interpreted independently?
7. Did increasing the number of epochs improve the generated images in your experiment? Support your answer with evidence.
8. Did changing the learning rate or latent dimension improve the results? Explain.
9. What is mode collapse? Identify any signs of it in your output.
10. What changes would be necessary to generate a specific clothing category rather than an arbitrary clothing image?